Topic: Terraform Modules — Root vs Child, `module` Block, Local / GitHub / Registry Sources, Passing Inputs

(Your notes, cleaned up from dictation + screenshots — Lecture 9)

Quick recall

> **Definition (lecture):** A **Terraform module** is a collection of standard configuration files in a dedicated directory. **Terraform modules are the main feature that allows us to reuse resource definitions across multiple projects, or simply have a better organization in a single project.** This is much like standard programming: instead of a single file containing all the code, we organize our code across multiple files and packages.
>
> **Definition (reference):** A Terraform module is **a self-contained collection of multiple Terraform configuration files stored together in a single directory**. It acts as a **reusable building block** that abstracts and groups related infrastructure resources into a single logical unit, working similarly to **functions** in programming languages.

In simple terms: "*Modules allow you to group resources together and reuse this group later, possibly many times.*"

This is one of the **most important** Terraform topics, because **modules are used in all production environments**. The purpose of a module is **reusability of code**.

---

1. Why modules? Reusability explained

Suppose we want to create a **web application on AWS**. It has several layers, and no single person works on everything:

Layer	Team	Resources
Front end	Front-end / compute team	EC2 instances, ASG, ALB…
Back end	Storage / data team	S3, EBS, EFS…
Network	Network team	VPC, subnets, security groups, NACLs…

Each team writes Terraform code for this project. Next time, someone works on **another project**, and they weren't part of the team that built that code. Without modules, they'd have to **write it all again from scratch**.

With modules: the code another team already built for creating resources is **reused**. You call it, **change the values** for the new project's requirements, and it serves the purpose. Nobody has to develop the Terraform files from scratch each time. **That's reusability: accessing already built code.**

Where can that shared code come from?

1. **Locally**: files on your own system or server (or SCP'd / copied from someone else).
2. **GitHub** (any Git repository).
3. **HTTPS URL** (e.g. a zip/tar archive on a web server).
4. **The official Terraform Registry** (`registry.terraform.io`).

---

2. Terminology: module, root module, child module

1. **Module:** a set of Terraform configuration files **in a single directory**. Even a simple configuration of a single directory with one or more `.tf` files is a module. **So every Terraform configuration is part of a module.**
2. **Root module:** the directory **where we run Terraform commands** is the root module. It's *our* main code.
3. **Child module:** a module **called by another module** (by the root). It's code coming from another source: a local subfolder, GitHub, HTTPS or the official registry.

Example 1: directory A and B

```
A/                      ← ROOT module  (we run terraform here)
├── main.tf
└── B/                  ← CHILD module (only if A calls it with a module block)
    ├── vpc.tf
    └── ...
```

Example 2: the standard layout

```
tf-module-example/
├── main.tf                    ← ROOT module
└── modules/
    ├── server-module/         ← CHILD module
    │   ├── main.tf
    │   ├── output.tf
    │   └── variables.tf
    └── storage-module/        ← CHILD module
```

Calling the folder `modules/` isn't required (you could call it anything), but **the standard says keep the name `modules`**.

A full project typically looks like this:

```
TERRAFORM/
├── s3-website-hosting/    ┐ child modules
├── vpc/                   ┘
├── LICENSE                ┐
├── main.tf                │
├── outputs.tf             │ root module
├── README.md              │
├── terraform.tfvars       │
└── variables.tf           ┘
```

> 📝 **Precise version:** a subfolder isn't a child module just because it sits under the root folder. Terraform only reads `.tf` files in the **current directory**. A subfolder becomes a child module **only when the root calls it with a `module` block**. Until then it's ignored (see §4).

---

3. The `module` block syntax

```
module "<NAME>" {
  source = "<SOURCE>"     # where the module code is: local path, GitHub, HTTPS, registry
  [CONFIG ...]            # input variables of the module (+ optional version, count, provid
}
```

- `<NAME>` is a **local label** for this call (like a resource name). It's used in addresses: `module.<NAME>.<resource>` and `module.<NAME>.<output>`.
- `source` **is the only required argument.**

- Every other argument maps to a `variable` **declared inside the module**. That's how you "change the values" for your project.

---

4. Lab 1: local child module (EC2)

Build the directory structure

```
[root@Devops-server ~]# mkdir project1
[root@Devops-server ~]# cd project1/
[root@Devops-server project1]# mkdir modules
[root@Devops-server project1]# cd modules/
[root@Devops-server modules]# mkdir ec2_modules vpc_modules
```

Root module code: `project1/s3.tf`

```
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = {
    Name = "my_buck"
  }
}
```

Child module code: `project1/modules/ec2_modules/`

```
# varibale.tf   (file name typo is harmless — Terraform reads any *.tf)
variable "instance_count" {
  type    = number
  default = 1
}
```

```
# ec2.tf
resource "aws_instance" "linux" {
  count         = var.instance_count
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"

  tags = {
    Name = "web-server-${count.index}"
  }
}
```

```
[root@Devops-server project1]# tree .
.
├── modules
│   ├── ec2_modules
│   │   ├── ec2.tf
│   │   └── varibale.tf
│   └── vpc_modules
└── s3.tf

3 directories, 3 files
```

Overall, `s3.tf` **is the root module** (it belongs to `project1`). `ec2_modules` **and** `vpc_modules`, and the files under them, are **child modules** under `modules/`.

The scenario

We're good at writing S3 code, and we've put it in our project directory. Now our manager asks us to **write code for EC2 as well**, and we don't know what to write. Somebody **shares the** `modules` **folder** with the code already created for another project, and we copy it into `project1`.

But copying alone does nothing. If we run `terraform init` and `terraform apply` from `project1`, it creates **only the S3 bucket, not the EC2 instance**. Terraform runs only the root module. **It doesn't call the child module on its own.** To call it, we write a `module` block.

Calling the child module (and changing a value)

```
# s3.tf
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = {
    Name = "my_buck"
  }
}

module "my_module" {
  source         = "./modules/ec2_modules"
  instance_count = 2      # Passing variable to control number of EC2 instances inside the m
}
```

- `source` = the **path** where the code is: `modules/ec2_modules` (local paths must start with `./` or `../`).
- In the module, `instance_count` defaults to **1**. Here we **override it to 2**. That's the "reuse the code, just change the values" idea in action.

`terraform plan` → "Module not installed"

```
[root@Devops-server project1]# terraform plan

Error: Module not installed

  on s3.tf line 7:
   7: module "my_module" {

This module is not yet installed. Run "terraform init" to install all modules required by th
```

Whenever you **add or change a `module` block** (or its `source`), you must **initialize again**.

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Initializing modules...
- my_module in modules/ec2_modules
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.52.0

Terraform has been successfully initialized!
```

After `apply`, you get the bucket plus **2 instances**, `web-server-0` and `web-server-1`, addressed as `module.my_module.aws_instance.linux[0]` and `[1]`.

> **This is how we use the code. It's called reusability. We don't have to write the code, we reuse it.**

---

5. Lab 2: a complete VPC module (≈110 lines) called with new CIDRs

Somebody shared the code to **create a whole VPC**. It goes in `project1/modules/vpc_modules/vpc.tf`:

```
provider "aws" {
  region = "ap-south-1"
}
# Define variables Here
variable "vpc_cidr" {
  default     = "10.1.0.0/16"
  description = "cidr for our custom vpc"
}
variable "subnet_cidr" {
  default     = "10.1.1.0/24"
  description = "cidr for subnet"
}
variable "availability_zone" {
  default     = "ap-south-1a"
  description = "AZ for subnet"
}
variable "instance_ami" {
  default     = "ami-0cca134ec43cf708f"
  description = "default ami for instances"
}
variable "instance_type" {
  default     = "t2.micro"
  description = "instance type for ec2"
}
variable "env_tag" {
  default     = "production"
  description = "environment tag"
}

# code - creating vpc
resource "aws_vpc" "vpcone" {
  cidr_block = "${var.vpc_cidr}"
  tags = { Name = "${var.env_tag}" }
}

# code - creating IG and attaching it to VPC
resource "aws_internet_gateway" "vpcone-ig" {
  vpc_id = "${aws_vpc.vpcone.id}"
  tags   = { Name = "${var.env_tag}" }
}

# code - create subnet inside our vpc
resource "aws_subnet" "subnet_public" {
  vpc_id                  = "${aws_vpc.vpcone.id}"
  cidr_block              = "${var.subnet_cidr}"
  map_public_ip_on_launch = "true"
  availability_zone       = "${var.availability_zone}"
  tags = { Name = "${var.env_tag}" }
}
```

```
# code - modifying route
resource "aws_route_table" "rtb_public" {
  vpc_id = "${aws_vpc.vpcone.id}"
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = "${aws_internet_gateway.vpcone-ig.id}"
  }
  tags = { Name = "${var.env_tag}" }
}

# code - attaching subnets to route table
resource "aws_route_table_association" "rta_subnet_public" {
  subnet_id      = "${aws_subnet.subnet_public.id}"
  route_table_id = "${aws_route_table.rtb_public.id}"
}

# code - create security group
resource "aws_security_group" "sg_newvpc" {
  name   = "newvpc"
  vpc_id = "${aws_vpc.vpcone.id}"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = { Name = "${var.env_tag}" }
}

# code - create instance
resource "aws_instance" "test" {
  ami                    = "${var.instance_ami}"
  instance_type          = "${var.instance_type}"
  subnet_id              = "${aws_subnet.subnet_public.id}"
  vpc_security_group_ids = ["${aws_security_group.sg_newvpc.id}"]
  tags = { Name = "${var.env_tag}" }
}
```

What it builds: **VPC → Internet Gateway → public subnet (public IP on launch) → route table with `0.0.0.0/0` via the IGW → route-table association → SG (SSH in, all out) → EC2 instance in that subnet**. Implicit dependencies (Chapter 02) order everything automatically.

Call it from the root, changing only the CIDRs

```
# s3.tf
module "vpc-module" {
  source      = "./modules/vpc_modules"
  vpc_cidr    = "192.168.0.0/16"
  subnet_cidr = "192.168.1.0/24"
}
```

In the code we just **change the VPC CIDR and subnet CIDR** to suit our needs. Everything else (AZ, AMI, instance type, `env_tag = "production"`) uses the module's **defaults**. Then `terraform init` (new module) → `plan` → `apply`.

> ⚠️ **Review notes on this shared module** (it works, but it's not how you'd write it today):
>
> - `provider "aws"` **inside a child module** is a legacy pattern. Providers belong in the **root module**, and children inherit them. A module with its own provider block **can't use `count` / `for_each` / `depends_on`**. If you later delete the `module` block, Terraform errors, because the provider config needed to destroy those resources is gone.
> - `"${var.vpc_cidr}"` is old interpolation syntax. Write `var.vpc_cidr` and `aws_vpc.vpcone.id` (Chapter 06).
> - `map_public_ip_on_launch = "true"` should be the bool `true`.
> - Variables have no `type`. Add `type = string` so callers get early errors.
> - There's no `outputs.tf`, so the root **can't read** the VPC ID, subnet ID etc. (see §8).
> - SSH open to `0.0.0.0/0` is lab-only.

---

6. Lab 3: module from GitHub (remote)

So far the module code was **local**. Now the code is in a **GitHub repository**: **`github.com/srm71118/adamrepo`** (public, branch `main`) with two files:

```
# variable.tf  — Server Module Variables
variable "instance_name" {
  description = "Instance Name"
  default     = ""
}
variable "ami" {
  description = "Go with yur AMI ID"
  default     = "ami-xxxx"
}
variable "instance_type" {
  description = "Instance Type"            # no default → caller MUST pass it
}
variable "key" {
  description = "Go with yur aws key pair"
  default     = "ami-xxxx"                 # (copy-paste slip in the repo; harmless because
}
```

```
# ec2.tf
provider "aws" {
  region = "ap-south-1"
}

## Creating a AWS EC2 Instance
resource "aws_instance" "server-instance" {
  # Instance Configuration
  ami           = var.ami
  instance_type = var.instance_type
  key_name      = var.key
  # Instance Tags
  tags = {
    Name = "${var.instance_name}"
  }
}
```

In the root, we only change the `source` section:

```
# s3.tf
resource "aws_s3_bucket" "first" {
  bucket = "ramprateek"
  tags = { Name = "my_buck" }
}

module "adam-module" {
  source        = "github.com/srm71118/adamrepo"
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  instance_name = "Demo-server"
  key           = "ram_aws"
}
```

> ⚠️ The first draft in the editor had an **extra** `}` after the module block (line 15). Terraform would fail with "*Argument or block definition required*". Keep braces balanced, and `terraform fmt` / `terraform validate` will catch it.

`terraform init` **clones the repo** into `.terraform/modules/adam-module`. Then `apply`:

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_s3_bucket.first: Creating...
module.adam-module.aws_instance.server-instance: Creating...
aws_s3_bucket.first: Creation complete after 2s [id=ramprateek]
```

Look at the **resource address**: `module.adam-module.aws_instance.server-instance`. Resources inside a child module are always prefixed with `module.<name>.`.

Pin the version in real code so a push to `main` can't change your infrastructure unexpectedly:

```
source = "git::https://github.com/srm71118/adamrepo.git?ref=v1.0.0"   # tag, branch or commi
```

---

7. Lab 4: module from the official Terraform Registry

Instead of taking modules from other people, we can use the **official Terraform Registry**, **registry.terraform.io** (at the time: **6,795 providers, 22,408 modules & counting**). Browse Modules → e.g. `terraform-aws-modules/s3-bucket/aws`.

Registry source format: `<NAMESPACE>/<NAME>/<PROVIDER>`.

```
# main.tf
module "s3_bucket" {
  source = "terraform-aws-modules/s3-bucket/aws"

  bucket = "lalalalal123212344"
  acl    = "private"

  control_object_ownership = true
  object_ownership         = "ObjectWriter"

  versioning = {
    enabled = true
  }
}
```

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Initializing modules...
Downloading registry.terraform.io/terraform-aws-modules/s3-bucket/aws 5.14.1 for s3_bucket..
- s3_bucket in .terraform/modules/s3_bucket
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.52.0

Terraform has been successfully initialized!
...
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory.
```

The whole module is downloaded under `.terraform/modules/`, and Terraform uses it from there when creating the resources:

```
[root@Devops-server project1]# ls .terraform/modules/
adam-module  modules.json  s3_bucket
```

Entry	What it is
`adam-module/`	the GitHub module, cloned
`s3_bucket/`	the registry module, version **5.14.1** downloaded
`modules.json`	Terraform's index: module key → source → version → local dir

*Local modules (`./modules/...`) are **not** copied. `modules.json` just points to their real path.*

> 📝 **Always pin registry versions.** Without `version`, `init` picks the **latest** release (5.14.1 here), and a future major version can break your code:
>
> ```
> module "s3_bucket" {
>   source  = "terraform-aws-modules/s3-bucket/aws"
>   version = "~> 5.14"          # any 5.x ≥ 5.14, never 6.0
>   ...
> }
> ```
>
> The `version` argument works **only for registry sources**. For Git, use `?ref=`.

Clean-up seen in the lab

```
Destroy complete! Resources: 2 destroyed.
[root@Devops-server project1]# rm -rf *
[root@Devops-server project1]# vim main.tf
[root@Devops-server project1]# terraform init
Downloading registry.terraform.io/terraform-aws-modules/s3-bucket/aws 5.14.1 for s3_bucket..
- Reusing previous version of hashicorp/aws from the dependency lock file
```

Note: `rm -rf *` **doesn't delete hidden files.** `.terraform/` and `.terraform.lock.hcl` survive, which is why init says "*Reusing previous version … from the dependency lock file*". For a truly clean folder: `rm -rf .terraform .terraform.lock.hcl`.

---

8. Module inputs and outputs: the "function" view



- **Inputs** = `variable` blocks in the child. The caller sets them as arguments in the `module` block. Variables with **no default are required** (like `instance_type` in the GitHub module).
- **Outputs** = `output` blocks in the child. **The only way** the root can read values from inside a module:

```
# modules/vpc_modules/outputs.tf
output "vpc_id"    { value = aws_vpc.vpcone.id }
output "subnet_id" { value = aws_subnet.subnet_public.id }

# root
module "app" {
  source    = "./modules/ec2_modules"
  subnet_id = module.vpc-module.subnet_id     # wire one module into another
}
output "vpc_id" { value = module.vpc-module.vpc_id }
```

- Resources **inside** a module can't be referenced directly from the root (`aws_vpc.vpcone.id` won't work there). Only `module.<name>.<output>` can be.

Module source types (cheat sheet)

Source	Example	Version pinning
Local path	`./modules/ec2_modules`	n/a (same repo)
Terraform Registry	`terraform-aws-modules/vpc/aws`	`version = "~> 5.0"`
GitHub (shorthand)	`github.com/srm71118/adamrepo`	`?ref=v1.0.0`
Generic Git (HTTPS/SSH)	`git::https://example.com/net.git//vpc?ref=v2.1.0`	`?ref=` tag/branch/SHA
HTTPS archive	`https://example.com/modules/vpc.zip`	in the file name/URL
S3 / GCS bucket	`s3::https://s3.amazonaws.com/bucket/vpc.zip`	in the object key
Private registry	`app.terraform.io/acme/vpc/aws`	`version = "1.2.0"`

*(`//vpc` after a repo URL = a **subdirectory** inside that repo.)*

---

9. Production best practices

- **Standard module layout:** `main.tf` , `variables.tf` (with `type` + `description`), `outputs.tf` , `versions.tf` (`required_providers` + `required_version`), and `README.md` .
- **No `provider` blocks in reusable modules.** Declare `required_providers` only, and let the root configure them (pass aliases with `providers = { aws = aws.mumbai }`).

- **Pin everything:** registry `version`, Git `?ref=` tags. Upgrade deliberately with `terraform init -upgrade`.
- **Keep modules small and focused** (network, compute, storage). Avoid one giant "everything" module.
- **Multiple copies:** `count` / `for_each` work on module blocks too, e.g. `for_each = toset(["dev","prod"])` → `module.vpc["dev"]`.
- **Refactor safely:** moving resources into a module changes their address. Use a `moved {}` block so Terraform doesn't destroy and recreate them:

```
moved {
  from = aws_instance.linux
  to   = module.my_module.aws_instance.linux
}
```

- **Test & document:** `terraform validate`, `terraform fmt -recursive`, `terraform test`, and `terraform-docs` to generate the README.

Command reference (this chapter)

```
mkdir -p project1/modules/{ec2_modules,vpc_modules}   # standard layout
tree .                                                # view root + child module folders
terraform init                                        # REQUIRED after adding/changing a mod
terraform init -upgrade                               # fetch newer module/provider versions
terraform get                                         # download/update modules only
terraform plan                                        # "Module not installed" → you forgot
terraform state list                                  # module resources: module.<name>.<typ
terraform state show 'module.my_module.aws_instance.linux[0]'
terraform output                                      # root outputs (re-export module outpu
terraform plan -target=module.vpc-module              # plan just one module (troubleshootin
terraform fmt -recursive                              # format root + all child module folde
terraform validate                                    # catches the stray "}" / bad argument
ls .terraform/modules/                                # downloaded remote modules + modules.
rm -rf .terraform .terraform.lock.hcl                 # truly clean (rm -rf * skips hidden f
```

Still to come

- Writing a module with full `outputs.tf` and chaining modules (VPC → EC2 → ALB)
- `for_each` on modules (one module, many environments)
- Publishing modules to a private registry / HCP Terraform

- `terraform test` for modules